VS Code Workbench README

gemstone-js Workbench

VS Code wrapper for the gemstone-js Explorer. The extension starts the local Explorer server, embeds it in a webview, exposes connection/object trees, and provides basic GemStone code evaluation and debugging commands.

Commands

- `GemStone: Open Explorer` starts the JS Explorer server and opens it in a VS Code webview.
- `GemStone: Open Explorer in Browser` opens the Explorer URL in the system browser.
- `GemStone: Copy Explorer URL` copies the managed Explorer URL.
- `GemStone: Copy Connection Summary` copies a redacted connection summary for diagnostics.
- `GemStone: Copy Doctor Report` runs Doctor and copies a redacted diagnostics report.
- `GemStone: Open Class Browser` opens the Explorer Class Browser focused on a class from a tree row, editor selection, or prompt.
- `GemStone: Open Workspace`, `GemStone: Open Globals`, `GemStone: Open Roots`, `GemStone: Open Symbol List Browser`, `GemStone: Open Codegen`, and `GemStone: Open Status Log` open focused Explorer tool windows.
- `GemStone: Inspect OOP` opens the Explorer object inspector focused on an OOP from a tree row, editor selection, or prompt.
- `GemStone: Copy OOP` copies an object OOP from tree context menus, editor selection, or a prompt.
- `GemStone: Copy Object Name` copies an OOP-backed tree row name from root/global entries or a prompt.
- `GemStone: Copy Class Name` copies a class name from the Classes tree or a prompt.
- `GemStone: Start Explorer Server` starts the Explorer without opening a UI.
- `GemStone: Stop Explorer Server` stops the managed Explorer process from the command palette or Connection tree.
- `GemStone: Restart Explorer Server` restarts the managed Explorer process.
- `GemStone: Filter Roots`, `GemStone: Filter Globals`, and `GemStone: Filter Classes` narrow tree contents.
- `GemStone: Clear Tree Filters` clears Roots, Globals, and Classes filters together.
- `GemStone: Doctor` runs connection/config diagnostics and writes redacted JSON to the GemStone JS output channel.
- `GemStone: Open Output` opens the GemStone JS output channel.
- `GemStone: Evaluate Selection` sends the current selection to the Explorer session. If evaluation raises, the debugger opens automatically.
- `GemStone: Evaluate Selection As...` evaluates the current selection with a one-off inspect, value, or OOP return kind.
- `GemStone: Debug Selection` starts a VS Code debug session for the current selection.
- `GemStone: Debug Selection As...` debugs the current selection with a one-off inspect, value, or OOP return kind.
- `GemStone: Debug File` starts a VS Code debug session for the current editor contents.
- `GemStone: Debug File As...` debugs the current editor contents with a one-off inspect, value, or OOP return kind.
- `GemStone: Run File` evaluates the current editor contents.
- `GemStone: Run File As...` evaluates the current editor contents with a one-off inspect, value, or OOP return kind.
- `GemStone: Configure Connection` prompts for the core connection settings and stores the password in

SecretStorage.

- `GemStone: Set Explorer Open Mode` switches between VS Code webview and external browser opening.
- `GemStone: Set Native Session Worker` enables or disables `GS_NATIVE_SESSION_WORKER` and stops the Explorer so the next launch uses the new setting.
- `GemStone: Set Default Return Kind` updates `gemstoneJs.defaultReturnKind` from a quick pick.
- `GemStone: Set Password` stores the GemStone password in VS Code SecretStorage.
- `GemStone: Clear Password` removes the SecretStorage password.
- `GemStone: Open Settings` opens the extension settings filtered to `gemstoneJs`.

Views

The GemStone activity bar contributes:

- Connection status
- Roots
- Globals
- Classes

Roots, globals, and classes load from the Explorer API. OOP-backed items can be inspected from the tree. Class rows open the Explorer Class Browser focused on that class and expose context menu actions for opening/copying the class name. The command palette can also open a selected class name or prompt for one. OOP rows open the Explorer object inspector and expose context menu actions for inspect, copy OOP, and copy object name; command-palette OOP actions can use a selected decimal OOP before prompting. Root/global/class views have their own refresh and filter actions, plus a shared clear-filters command.

Language Support

The extension contributes the `smalltalk` language id for `.st`, `.gs`, and `.topaz` files, with lightweight GemStone Smalltalk syntax highlighting and editor pairs for strings, comments, blocks, arrays, and braces. It also provides snippets for common method, collection, exception, cleanup, globals, and debug expression patterns.

Settings

Configure `gemstoneJs.*` in VS Code settings:

- `gemstoneJs.repoPath`
- `gemstoneJs.nodePath`
- `gemstoneJs.explorerScriptPath`
- `gemstoneJs.explorerHost`
- `gemstoneJs.explorerPort`
- `gemstoneJs.openMode`
- `gemstoneJs.user`
- `gemstoneJs.password`
- `gemstoneJs.stone`
- `gemstoneJs.netldiHost`
- `gemstoneJs.netldiNameOrPort`
- `gemstoneJs.gemService`
- `gemstoneJs.nativeSessionWorker`
- `gemstoneJs.extraEnv`
- `gemstoneJs.defaultReturnKind`

The extension maps these settings to the Explorer environment, including `GS_USER`, `GS_PASS`, `GS_STONE`, `GS_NETLDI_HOST`, `GS_NETLDI_NAME_OR_PORT`, and `GS_NATIVE_SESSION_WORKER`. Prefer `GemStone: Set Password` over the legacy `gemstoneJs.password` setting so the password is stored in VS Code SecretStorage instead of plain settings JSON.

Debugger

The `gemstone-js` debug type wraps the Explorer debugger API. It supports:

- Stack display
- Source previews from GemStone context frames
- Local/receiver/exception variables
- Continue
- Step over
- Step in
- Step out
- Restart

The extension contributes launch configuration snippets for debugging the current selection or an inline Smalltalk expression. Evaluate, Debug Selection, Debug File, and Run File commands are available from the editor title area and editor context menu. The `As...` variants are available from the editor context menu for one-off return-kind selection.

The debugger is intentionally thin in this first version; session semantics remain owned by the Explorer server.

Development

From this directory:

```
npm run verify
```

This checks the extension entrypoint, packages a versioned VSIX, and verifies the archive contents. It also runs an offline smoke test that activates the extension against a lightweight VS Code API mock. VSIX verification checks required files, the packaged extension manifest, command/view/debugger contributions, and optional checksum artifacts.

To run the real VS Code extension-host smoke test:

```
GS_RUN_VSCODE_HOST=1 npm run test:host
```

That test starts VS Code through `@vscode/test-electron`, activates the extension, opens the Explorer against a fake local Explorer server, and starts/stops a `gemstone-js` debug session without requiring a live Stone.

To package a fixed local smoke-test artifact:

```
npm run package:dry-run
```

To produce the release artifact and checksum used by CI:

```
npm run release:package
```

Before bumping the VSIX version, add the matching entry to `CHANGELOG.md`. Then run:

```
npm run release -- 0.1.1
```

To publish from a prepared checkout with a Marketplace token:

```
VSCE_PAT=... npm run release:publish -- 0.1.1
```

The GitHub `VS Code Workbench` workflow builds and uploads the VSIX on changes under this directory. Run it manually with `publish-to-marketplace=true` to publish the package with the repository `VSCE_PAT` secret.